

PP_01 | HelloWorld

1. From <https://repo.anaconda.com/archive/> download an installation file of Anaconda Navigator for your OS, architecture, and installation file (Win .exe, Linux .sh, or MacOSX .pkg) and install

← → ↻ repo.anaconda.com/archive/ ☆ 🔒 🌐 📁 👤 ⋮

Index of /archive

By continuing installation, you hereby acknowledge and assent to the legal terms applicable to the Anaconda Distribution Installer at anaconda.com/legal.

Filename	Size	Last Modified	SHA256
Anaconda3-2025.12-2-Windows-x86_64.exe	1.1G	2026-01-13 11:06:18	2e0b8e40ec7600793f116250f5c1775c866
Anaconda3-2025.12-2-MacOSX-arm64.sh	880.9M	2026-01-13 11:06:18	8d0b858358456d4ee159feb0c4ee6d63559
Anaconda3-2025.12-2-MacOSX-arm64.pkg	878.8M	2026-01-13 11:06:18	d2b4b7445dafd1202d6e3a0f76ba08f0470
Anaconda3-2025.12-2-Linux-x86_64.sh	1.2G	2026-01-13 11:06:18	57b2b48cc5b8665e25fce7011f0389d47c1
Anaconda3-2025.12-2-Linux-aarch64.sh	1.0G	2026-01-13 11:06:18	be13a20f31d82508a061984fa3555369cfd
Anaconda3-2025.12-1-Windows-x86_64.exe	1.1G	2025-12-09 13:48:52	09c8a69e7a717a963a9f251697473f4ea88
Anaconda3-2025.12-1-MacOSX-arm64.sh	878.9M	2025-12-09 13:48:52	f998f0918b9f06e08c3444f2b2c897fd303
Anaconda3-2025.12-1-MacOSX-arm64.pkg	876.8M	2025-12-09 13:48:52	ed8f2fe4d828eff3aeaccd16af6511c6d6
Anaconda3-2025.12-1-Linux-x86_64.sh	1.2G	2025-12-09 13:48:52	132f1f312d05e391906b959ad83aa411b97
Anaconda3-2025.12-1-Linux-aarch64.sh	1.0G	2025-12-09 13:48:52	9ba6b32919df1964377a4ac5c5e4de645b
Anaconda3-2025.06-1-Windows-x86_64.exe	914.6M	2025-07-18 13:52:27	36ba946abeb6bb7ba3235e0ccb67660eb5
Anaconda3-2025.06-1-MacOSX-x86_64.sh	802.9M	2025-07-18 13:52:27	58139fe195337f3041259e3a611339ed3af
Anaconda3-2025.06-1-MacOSX-x86_64.pkg	800.3M	2025-07-18 13:52:27	f1a9a495e06e94d83c779f01701426364b6
Anaconda3-2025.06-1-MacOSX-arm64.sh	783.9M	2025-07-18 13:52:27	f9366a024ac6f043dc224986c54f1b4e122
Anaconda3-2025.06-1-MacOSX-arm64.pkg	781.3M	2025-07-18 13:52:27	4f81547918274bfff6570d58ed773603dbd6
Anaconda3-2025.06-1-Linux-x86_64.sh	1.0G	2025-07-18 13:52:27	82976426a2c91fe1453281def386f9ebdbd
Anaconda3-2025.06-1-Linux-aarch64.sh	886.6M	2025-07-18 13:52:27	98dfb82732991f3f4d385a265da5fd2190d
Anaconda3-2025.06-0-Windows-x86_64.exe	914.3M	2025-06-23 10:37:31	00ec02d3db9a9a89002f005f5200c6557a
Anaconda3-2025.06-0-MacOSX-x86_64.sh	778.3M	2025-06-23 10:37:31	8625a155ff1d2848afa360e70357e14c25f
Anaconda3-2025.06-0-MacOSX-x86_64.pkg	775.9M	2025-06-23 10:37:31	7ecc4ccc2d5c828965a5a742d59fa7ef128
Anaconda3-2025.06-0-MacOSX-arm64.sh	756.7M	2025-06-23 10:37:31	195f234204e2f18803cea38bbebefcaac5a
Anaconda3-2025.06-0-MacOSX-arm64.pkg	754.1M	2025-06-23 10:37:31	79b21d8e0927d3e4cad1ea00be0e6cbe696
Anaconda3-2025.06-0-Linux-x86_64.sh	1.0G	2025-06-23 10:37:31	08db13f6db5dd4f99b3a6db96c68d064d50
Anaconda3-2025.06-0-Linux-aarch64.sh	886.2M	2025-06-23 10:37:31	e6177f844f6156f07978c85b1f18ca3b772
Anaconda3-2024.10-1-Windows-x86_64.exe	950.5M	2024-10-23 09:03:59	c1cb433e23997c84ade4ff7241b61b2f9b1
Anaconda3-2024.10-1-MacOSX-x86_64.sh	778.5M	2024-10-23 09:03:59	ad3eea1cc969e9df4d4571fc266aee06ec1
Anaconda3-2024.10-1-MacOSX-x86_64.pkg	776.0M	2024-10-23 09:03:59	dc1e2e123431edc1add68992b2db9db40fb
Anaconda3-2024.10-1-MacOSX-arm64.sh	747.2M	2024-10-23 09:03:59	f64ed797ce23ae1d07ead949bf6ff630b9
Anaconda3-2024.10-1-MacOSX-arm64.pkg	744.6M	2024-10-23 09:03:59	16436a4001b264ce51ca6c18c8cc90c6d00
Anaconda3-2024.10-1-Linux-x86_64.sh	1.0G	2024-10-23 09:03:59	3ba0a298155c32fbfd80cbc238298560bf6
Anaconda3-2024.10-1-Linux-s390x.sh	441.2M	2024-10-23 09:03:59	e00bd5e6c275695e8050a45aa85790315f5
Anaconda3-2024.10-1-Linux-aarch64.sh	843.2M	2024-10-23 09:03:59	489c608e8bdd2cf29dfbdd811cf99087cd
Anaconda3-2024.06-1-Windows-x86_64.exe	912.3M	2024-06-26 15:40:09	45ee9132b70e3d5a493c26429ddb4e2c9270a320971400c7291000606033a30
Anaconda3-2024.06-1-MacOSX-x86_64.sh	737.2M	2024-06-26 15:40:09	9b2aa48bfc58e337176d8b5f3776692815807e1346ec12ebcc46a01801bb4ee0
Anaconda3-2024.06-1-MacOSX-x86_64.pkg	734.7M	2024-06-26 15:40:09	9204bfd53baa9ae9a95cf15cbe0a11048c538163d7e50b7f0a2eece45a3f45f6
Anaconda3-2024.06-1-MacOSX-arm64.sh	707.3M	2024-06-26 15:40:09	f9e0795431910ee4ac341c0cc2a0b62e6c497440789b4117c20736db45b34204
Anaconda3-2024.06-1-MacOSX-arm64.pkg	704.7M	2024-06-26 15:40:09	5cba46cfbc9574247ef0d063b357e72fa58a88be7aedbb90600674f40992cf8f
Anaconda3-2024.06-1-Linux-x86_64.sh	1007.9M	2024-06-26 15:40:09	539bb43d9a52d758d0fdfa1b1b049920ec6f8c6d15ee9fe4a423355fe551a8f7

Install Anaconda3 2025.06-1

Installing Anaconda3 2025.06-1

- Introduction
- Read Me
- Licence
- Destination Select
- Installation Type
- Installation**
- Code in the Cloud
- Summary

Running package scripts...

Install time remaining: About 4 minutes

Go Back Continue



2. From within Anaconda Navigator, create a new environment using **Python 3.8.20**

The screenshot displays the Anaconda Navigator application interface. On the left sidebar, the 'Environments' tab is highlighted with a red box. The main panel shows a list of installed environments, with 'base (root)' selected. A 'Create new environment' dialog box is open in the center, featuring the following fields and options:

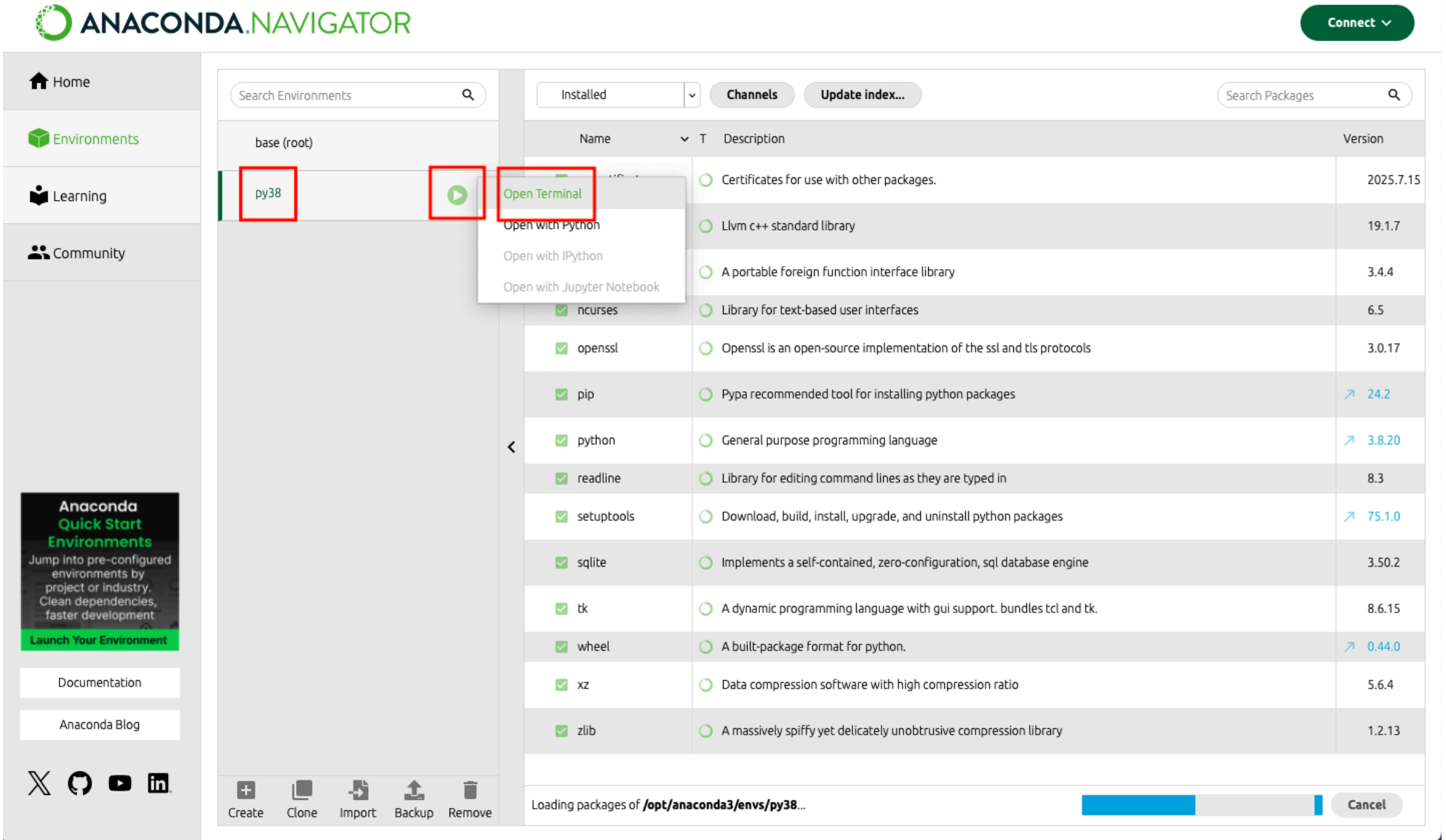
- Name:** 'py38' (highlighted with a red box)
- Location:** (empty field)
- Packages:**
 - ☒ Python 3.8.20 (highlighted with a red box)
 - ☐ R 3.6.1
- Buttons:** 'Cancel' and 'Create' (the 'Create' button is highlighted with a red box)

The background shows a table of installed packages with columns for Name, Description, and Version. The table lists various Anaconda extensions and their versions (4.20.0). At the bottom left, a 'Create' button in the sidebar is also highlighted with a red box.

Name	Description	Version
_anaconda_depends	Simplifies package management and deployment of anaconda	2025.06
aext-assistant	Anaconda extensions assistant library	4.20.0
aext-project-filebrowser-server	The aext-project-filebrowser-server component of anaconda-toolbox	4.20.0
aext-share-notebook	The aext-share-notebook component of anaconda-toolbox	4.20.0
aext-share-notebook-server	Anaconda extensions share notebook server	4.20.0
aext-shared	Anaconda extensions shared library	4.20.0
aext-toolbox		4.20.0
aext_environment...		4.20.0
aiobotocore	Async client for aws services using boto3 and aiohttp	2.19.0

562 packages available

3. From within the new environment, open a terminal



The screenshot shows the Anaconda Navigator application interface. On the left sidebar, the 'Environments' tab is selected. The main panel displays a list of environments, with 'py38' highlighted. A context menu is open over the 'py38' environment, showing options: 'Open Terminal', 'Open with Python', 'Open with IPython', and 'Open with Jupyter Notebook'. The 'Open Terminal' option is highlighted. The right panel shows a list of installed packages for the 'py38' environment, including 'certificates', 'libc++', 'libffi', 'ncurses', 'openssl', 'pip', 'python', 'readline', 'setuptools', 'sqlite', 'tk', 'wheel', 'xz', and 'zlib'. The status bar at the bottom indicates 'Loading packages of /opt/anaconda3/envs/py38...'.

ANACONDA.NAVIGATOR Connect

Home Environments Learning Community

Search Environments

base (root)

py38

Open Terminal

Open with Python

Open with IPython

Open with Jupyter Notebook

Installed Channels Update index...

Search Packages

Name	Description	Version
certificates	Certificates for use with other packages.	2025.7.15
libc++	Llvm c++ standard library	19.1.7
libffi	A portable foreign function interface library	3.4.4
ncurses	Library for text-based user interfaces	6.5
openssl	Openssl is an open-source implementation of the ssl and tls protocols	3.0.17
pip	Pypa recommended tool for installing python packages	24.2
python	General purpose programming language	3.8.20
readline	Library for editing command lines as they are typed in	8.3
setuptools	Download, build, install, upgrade, and uninstall python packages	75.1.0
sqlite	Implements a self-contained, zero-configuration, sql database engine	3.50.2
tk	A dynamic programming language with gui support. bundles tcl and tk.	8.6.15
wheel	A built-package format for python.	0.44.0
xz	Data compression software with high compression ratio	5.6.4
zlib	A massively spiffy yet delicately unobtrusive compression library	1.2.13

Create Clone Import Backup Remove

Loading packages of /opt/anaconda3/envs/py38... Cancel

Anaconda Quick Start Environments
Jump into pre-configured environments by project or industry. Clean dependencies, faster development.
[Launch Your Environment](#)

Documentation Anaconda Blog

X GitHub YouTube LinkedIn

4. Run these commands inside the terminal to install MPI and Python dependencies

Allowing/blocking the execution by the default of base environment when starting the terminal or power shell: `conda config --set auto_activate_base false`

Explore these commands in Terminal/Power Shell

`conda deactivate` (leaves the conda environment)

`conda activate` (activates the base environment)

`conda activate py38` (activates the conda environment with a specific Python version)

Install dependencies for parallel computation

`conda install mpi4py nb_conda_kernels ipykernel ipyparallel numpy jupyter`

`conda install conda-forge::matplotlib`

`conda install -c conda-forge openmpi` (this one seems to cause some issues on Windows, let's omit it for now)

5. Write the code below inside and save the file

Create a file in Windows PowerShell: `notepad hellompi.py -type file`

Create a file in MacOSX: `touch hello.mpi` and to edit `nano hellompi.py`
(you can also install the WSL)

<https://learn.microsoft.com/en-us/windows/wsl/install>

GNU nano 2.0.6 File: hellompi.py Modified

```

from mpi4py import MPI
import os

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()
pid = os.getpid()

message = "hello from rank: ", rank, " size ", size, " process ID: ", pid
print(message)

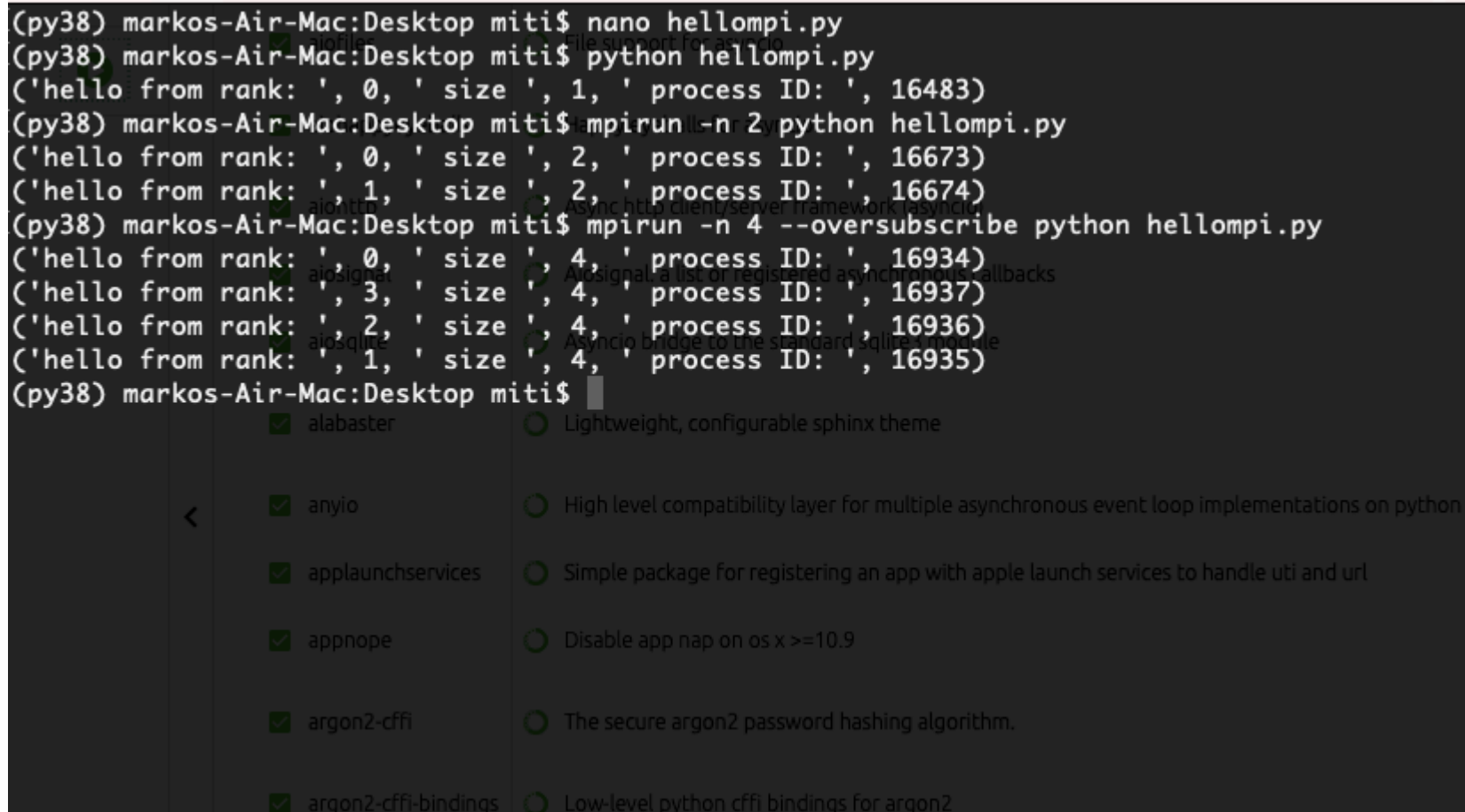
```

File Name to Write: hellompi.py

^G Get Help ^T To Files M-M Mac Format M-P Prepend
 ^C Cancel M-D DOS Format M-A Append M-B Backup File

6. Execute the file first as a regular Python script, then with mpi4py (using the mpirun command and experimenting with the **–oversubscribe** parameter). Change the number **n** on each run and observe the differences

```
(py38) markos-Air-Mac:Desktop miti$ nano hellompi.py
(py38) markos-Air-Mac:Desktop miti$ python hellompi.py
('hello from rank: ', 0, ' size ', 1, ' process ID: ', 16483)
(py38) markos-Air-Mac:Desktop miti$ mpirun -n 2 python hellompi.py
('hello from rank: ', 0, ' size ', 2, ' process ID: ', 16673)
('hello from rank: ', 1, ' size ', 2, ' process ID: ', 16674)
(py38) markos-Air-Mac:Desktop miti$ mpirun -n 4 --oversubscribe python hellompi.py
('hello from rank: ', 0, ' size ', 4, ' process ID: ', 16934)
('hello from rank: ', 3, ' size ', 4, ' process ID: ', 16937)
('hello from rank: ', 2, ' size ', 4, ' process ID: ', 16936)
('hello from rank: ', 1, ' size ', 4, ' process ID: ', 16935)
(py38) markos-Air-Mac:Desktop miti$
```



alabaster

anyio

applaunchservices

appnope

argon2-cffi

argon2-cffi-bindings

Lightweight, configurable sphinx theme

High level compatibility layer for multiple asynchronous event loop implementations on python

Simple package for registering an app with apple launch services to handle uti and url

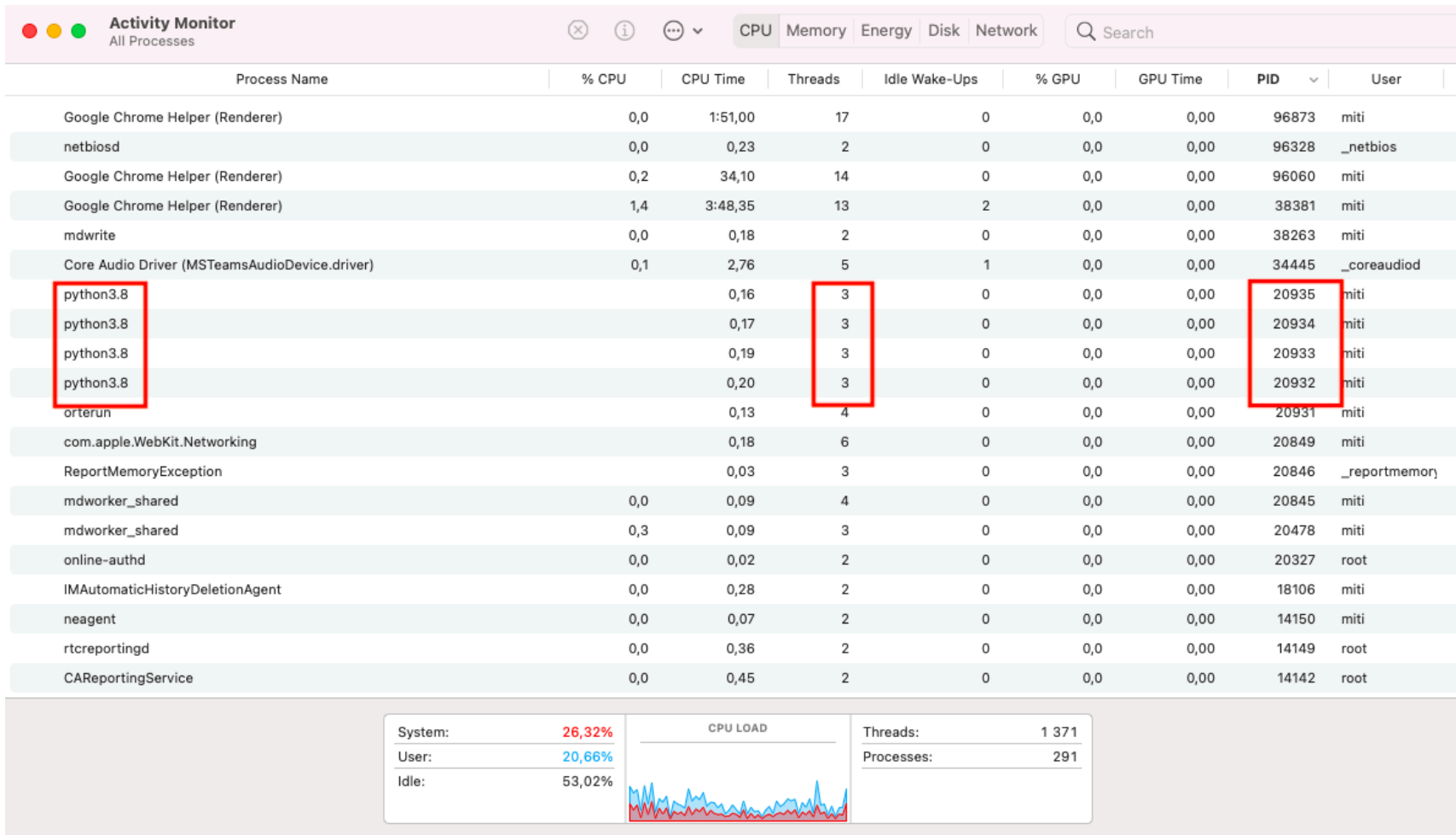
Disable app nap on os x >=10.9

The secure argon2 password hashing algorithm.

Low-level python cffi bindings for argon2

7. Add a while loop and counter to have additional computation time, and use Task Manager or Activity Monitor to see the running processes

```
(py38) markos-Air-Mac:Desktop miti$ mpirun -n 4 --oversubscribe python hellompi.py
('hello from rank: ', 3, ' size ', 4, ' process ID: ', 20935)
('hello from rank: ', 2, ' size ', 4, ' process ID: ', 20934)
('hello from rank: ', 0, ' size ', 4, ' process ID: ', 20932)
('hello from rank: ', 1, ' size ', 4, ' process ID: ', 20933)
```



PP_02 | Point to point Communication

PP_02_A | Send and Receive Python Objects



The `send()` and `recv()` methods provide for functionality to transmit data between two specific processes in the communicator group. It can be applied to any Python data object that can be pickled. The advantage is that the object is preserved, however it comes with the disadvantage that pickling the data takes more time than a direct memory copy. In the following example, an integer is transmitted from process 0 to process 1.

```
#!/usr/bin/env python
from mpi4py import MPI

# Communicator
comm = MPI.COMM_WORLD

# Get the rank of the current process in the communicator group
rank = comm.Get_rank()

# Variable to receive the data
data = None

# Process with rank 0 sends data to process with rank 1
if rank == 0:
    comm.send(42, dest=1)

# Process with rank 1 receives and stores data
if rank == 1:
    data = comm.recv(source=0)

# Each process in the communicator group prints its data
print(f'After send/receive, the value in process {rank} is {data}')
```


PP_02_B | Ping Pong

This example program uses the aforementioned `send()` and `recv()` methods to print a variable, `sendmsg`, depending on which rank the MPI program is presently working with.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
assert comm.size == 2

if comm.rank == 0:
    sendmsg = 777
    comm.send(sendmsg, dest=1, tag=55)
    recvmsg = comm.recv(source=1, tag=77)
else:
    recvmsg = comm.recv(source=0, tag=55)
    sendmsg = "abc"
    comm.send(sendmsg, dest=0, tag=77)
print(sendmsg)
```

Note how, at first line-by-line glance, the program's code sets `sendmsg` to 777 before it is set to abc. However, upon program execution, the output is abc first because of the dest and tag values. On rank 0 (during the program's initial stages), 777 is sent to destination 1. On rank 1 (remember there are only two ranks: 0 and 1), abc is sent to destination 0. The destination integers correspond to the ranks and the program leaves printing the `sendmsg` for last (after the `send()` and `recv()` methods have determined the variable values). This explains the output.

PP_02_C | Send and Recive Python Memory Objects

The following example illustrates the use of the uppercase versions of the methods `comm.Send()` and `comm.Recv()` to perform a transmission of data between processes from memory to memory. In our example we will again be sending a message between processors of rank 0 and 1 in the communicator group.

```
#!/usr/bin/env python
import numpy as np
from mpi4py import MPI

# Communicator
comm = MPI.COMM_WORLD

# Get the rank of the current process in the communicator group
rank = comm.Get_rank()

# Create empty buffer to receive data
buf = np.zeros(5, dtype=int)

# Process with rank 0 sends data to process with rank 1
if rank == 0:
    data = np.arange(1, 6)
    comm.Send([data, MPI.INT], dest=1)

# Process with rank 1 receives and stores data
if rank == 1:
    comm.Recv([buf, MPI.INT], source=0)

# Each process in the communicator group prints the content of its buffer
print(f'After Send/Receive, the value in process {rank} is {buf}')
```

PP_02_D | Non-blocking Send and Receive

MPI can also use non-blocking communications. This allows the program to send the message without waiting for the completion of the submission. This is useful for many parallel programs so we can overlap communication and computation while both take place simultaneously. The same can be done with receive, but if a message is not available and you do need the message, you may have to probe or even use a blocked receive. To wait for a message to be sent or received, we can also use the wait method, effectively converting the non-blocking message to a blocking one.

```
#!/usr/bin/env python
from mpi4py import MPI

# Communicator
comm = MPI.COMM_WORLD

# Get the rank of the current process in the communicator group
rank = comm.Get_rank()

# Variable to receive the data
data = None

# Process with rank 0 sends data to process with rank 1
if rank == 0:
    send = comm.isend(42, dest=1)
    send.wait()

# Process with rank 1 receives and stores data
if rank == 1:
    receive = comm.irecv(source=0)
    data = receive.wait()

# Each process in the communicator group prints its data
print(f'After isend/ireceive, the value in process {rank} is {data}')
```